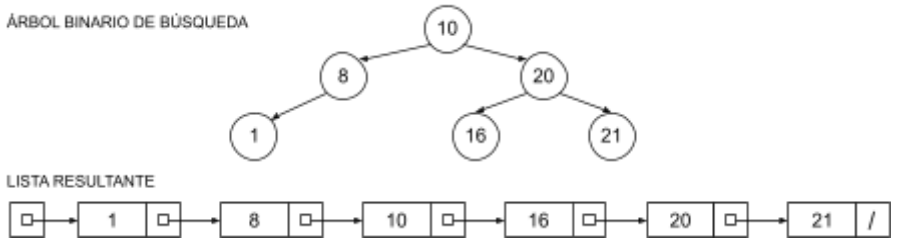
Nombre y apellido: _____

Ejercicios (puntaje)			Nota
1 (4 pts)	2 (4 pts)	3 (2 pts)	

CONSIDERACIONES: RESOLVER CADA EJERCICIO EN UNA HOJA DIFERENTE. LOS EJERCICIOS QUE NO ESTÉN CORRECTOS EN UN 50% NO SUMARÁN PUNTOS PARA LA NOTA FINAL. PARA CADA EJERCICIO DEBE **DEFINIR EL TIPO DE DATO** DE CADA TDA UTILIZADO. **EN TODOS LOS CASOS QUE UTILICE ESTRUCTURAS** QUE NO SEAN TDAs "ESTÁNDAR" **DEBE DEFINIR LOS STRUCTS**.

Ejercicio 1: SIN USAR TDA, desarrollar una función (y todas las necesarias) para **crear**, del modo más eficiente, **una lista dinámica** simplemente enlazada (SLL) con el valor (entero) **de todos los nodos un árbol binario de búsqueda** (SBT), teniendo en cuenta que la lista debe quedar **ordenada en forma ascendente**.
Ejemplo:



Encabezado de la función: `void crea_lista_de_sbt (sll** head, btn* root);`

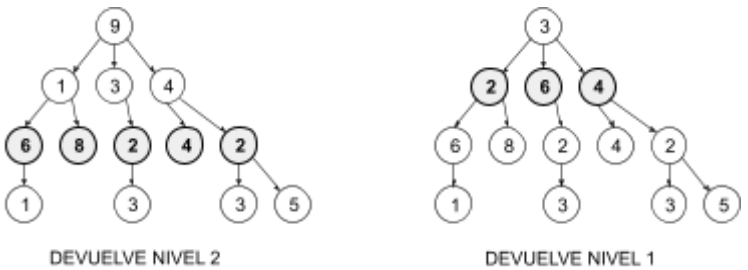
Ejercicio 2: Dado un **digrafo** (con implementación a elección, que puede o no utilizar TDAs), desarrollar una función (y todas las necesarias) para **completar un arreglo** (o vector) que contenga **la cantidad de vértices agrupados por grado**, que son **alcanzados por un vértice origen**. Es decir que el índice del arreglo/vector indicará el grado de los vértices, y el contenido la cantidad de vértices, siempre que los vértices puedan ser alcanzados directa o indirectamente por un vértice de origen.

Ejemplo: en el siguiente grafo, con origen en "***a***", se completa en el arreglo/vector en el índice 1: 2 por los nodos "***a***" y "***e***" de grado 1, en el índice 2: 2 por los vértices "***b***" y "***d***" de grado 2 y en el índice 3: 1 por el vértice "***c***" de grado 3 y "***f***" si bien es de grado 2 **NO SUMA** en el índice 2 porque **no es alcanzado** por "***a***".



Encabezado de la función: `void vertices_por_grado (graph* g, int origen, <t_struct>* a);`

Ejercicio 3: Desarrollar una función (y todas las necesarias) para que **dado un árbol n-ario devuelva el primer nivel en el cual todos los nodos son números pares**, o -1 si no se encuentra. Ejemplos:



Encabezado de la función: `int primer_nivel_par (ntn* root);`